

MODULE 6

Docker Avancé et Bonnes Pratiques

Support de cours professionnel & TP Pratique

Construction d'images, Optimisation, Sécurité & Multi-stage

THÉORIE	LAB PRATIQUE	DURÉE TOTALE
2 heures	3 heures	5 heures

Sofiene Khelif
ceo@karwisoft.com

1. POSITIONNEMENT DANS LE CURSUS

Progression Pédagogique Globale

Dans les modules précédents, les étudiants ont appris à manier les commandes fondamentales pour exécuter des conteneurs isolés (Module 4) et à orchestrer des architectures multi-services prêtes à l'emploi grâce à Docker Compose (Module 5).

Cependant, s'appuyer uniquement sur des images préexistantes publiées sur Docker Hub (telles que *nginx*, *postgres* ou *python*) ne permet pas de déployer des applications sur mesure développées en interne. Pour industrialiser leurs projets, les étudiants doivent franchir l'étape supérieure : passer de simples consommateurs d'images à concepteurs d'images personnalisées immuables et sécurisées.

Compétences Clés à Acquérir dans ce Module

Pour conteneuriser efficacement des applications métiers complexes, l'ingénieur DevOps doit maîtriser les sujets suivants :

- **Conception sur mesure :** Écrire un fichier `Dockerfile` propre, structuré et maintenable pour n'importe quelle pile logicielle.
- **Compréhension interne :** Analyser la mécanique des instructions de build et le rôle de chaque directive système.
- **Optimisation des performances :** Maîtriser le système de couches (layers) et exploiter intelligemment le cache de construction.
- **Réduction de l'empreinte :** Limiter drastiquement la taille des images finales pour accélérer les déploiements.
- **Durcissement sécuritaire :** Appliquer les principes de sécurité (utilisateur non-root, gestion des secrets, isolation).
- **Maintenance & Organisation :** Organiser le contexte de build avec `.dockerignore` et automatiser la chaîne avec Compose.

Enjeux Opérationnels pour l'Entreprise

L'absence de maîtrise de la création d'images personnalisées engendre des failles critiques dans la chaîne de valeur informatique : des pipelines de déploiement ralentis par des images pesant plusieurs gigaoctets, une instabilité de la production liée à l'absence de versions figées des dépendances, ainsi qu'une vulnérabilité accrue aux attaques réseaux dues à l'exécution en tant que super-utilisateur `root`.

Pourquoi l'industrialisation des images est cruciale en entreprise

En environnement de production, une image Docker mal conçue peut entraîner de graves problèmes opérationnels : temps de build extrêmement longs ralentissant les déploiements CI/CD, présence de vulnérabilités critiques non corrigées dans l'image de base, fuite d'identifiants de base de données intégrés par erreur dans les calques du conteneur, ou encore consommation excessive d'espace disque sur les clusters Kubernetes. Ce module apporte toutes les clés méthodologiques pour éviter ces écueils.

2. OBJECTIFS PÉDAGOGIQUES DÉTAILLÉS

À la fin de ce module intensif de 5 heures, l'étudiant sera parfaitement autonome pour :

- **Rédiger un Dockerfile fonctionnel** et optimisé pour une application Web (Python/Flask, Node.js, Go).
- **Construire, taguer et identifier** des images personnalisées via la CLI Docker (`docker build`).
- **Distinguer clairement** le rôle des instructions `RUN`, `CMD` et `ENTRYPOINT`.
- **Exploiter le cache de build** pour réduire le temps de compilation de plusieurs minutes à quelques secondes.
- **Exclure les fichiers inutiles et sensibles** en configurant un fichier `.dockerignore` rigoureux.
- **Différencier conceptuellement** la phase de construction statique (image) et la phase d'exécution dynamique (conteneur).
- **Sécuriser un conteneur** en créant et en forçant l'exécution sous un utilisateur non-privilégié.
- **Proscrire les mauvaises pratiques** (intégration de secrets, utilisation du mode `--privileged`).
- **Intégrer la directive `build:`** au sein de Docker Compose pour un workflow de dev/prod fluide.
- **Diagnostiquer et résoudre** rapidement les erreurs de build et de démarrage d'un conteneur.

Méthodologie Pédagogique et Approche Pratique

Le cours s'appuie sur une démarche très concrète : chaque concept théorique présenté est immédiatement suivi d'exemples de code Dockerfile, d'illustrations de cas d'erreurs réels observés en entreprise et d'exercices de refactoring. L'apprentissage est axé sur la compréhension fine des mécanismes internes de Docker plutôt que sur le simple apprentissage par cœur de commandes CLI.

Modalités d'Évaluation de l'Apprentissage

Les étudiants seront évalués en continu à travers la résolution de dysfonctionnements de build (troubleshooting), l'optimisation progressive d'un projet mal configuré et la réalisation finale d'un mini-projet individuel soumis aux contraintes de la production.

Structure et Découpage du Module (5 Heures)

- **PARTIE 1 — THÉORIE (2h) :** Principes du Dockerfile, gestion du cache, mécanismes de sécurité, optimisation des images et intégration Docker Compose.
- **PARTIE 2 — TRAVAUX PRATIQUES (3h) :** TP guidé pas à pas (création d'une API, optimisation, sécurité), exercice autonome et réalisation d'un mini-projet "Production-Ready".

3. POURQUOI CRÉER SES PROPRES IMAGES ?

Dans un cycle de développement logiciel moderne, l'image officielle fournie par la communauté (ex: `python:3.12-slim`) n'embarque que l'interpréteur de base. Elle ne contient ni le code source de l'entreprise, ni ses configurations spécifiques, ni ses dépendances tierces.

Structure d'une Image Personnalisée Produite en Entreprise

```
Image Personnalisée (Projet API)
├── 1. Environnement système & Interpréteur (ex: Python 3.12 Slim)
├── 2. Dépendances de l'application (ex: Flask, Gunicorn, SQLAlchemy)
├── 3. Code source métier (ex: app.py, modules, templates)
└── 4. Configuration de démarrage & Variables d'environnement (CMD / ENTRYPOINT)
```

Le Principe Fondamental de Reproductibilité

Créer sa propre image Docker garantit le principe de **reproductibilité stricte** :

- **Alignement des environnements :** L'image exécutée sur le poste du développeur est rigoureusement identique à celle testée en intégration continue (CI) et déployée sur les serveurs de production.
- **Élimination du syndrome "Ça marche sur ma machine" :** Toutes les dépendances système et logicielles sont figées et empaquetées dans un artefact immuable.
- **Traçabilité et Versionnage :** Chaque version d'image est associée à un tag précis (ex: `v1.2.0`) correspondant à un commit Git exact.
- **Déploiements atomiques :** La mise en production consiste simplement à remplacer un conteneur basé sur l'ancienne version par un nouveau conteneur basé sur la nouvelle image, sans intervention manuelle sur le serveur hôte.

L'Artefact Immuable comme Socle du DevOps

L'image Docker personnalisée constitue le véritable artefact de livraison de l'équipe de développement. Une fois compilée, testée et validée par les outils de CI/CD, elle est stockée dans un registre privé (Docker Registry, Nexus, Harbor) et ne doit plus jamais être modifiée manuellement. Les configurations spécifiques aux environnements (dev, staging, prod) y sont injectées au moment précis du lancement du conteneur.

4. QU'EST-CE QU'UN DOCKERFILE ? (ANATOMIE)

Un **Dockerfile** est un fichier texte d'instructions séquentielles indiquant à l'outil Docker comment construire automatiquement une image personnalisée pas à pas.

```
# Exemple de Dockerfile pour une API Python Flask FROM python:3.12-slim WORKDIR /app COPY
requirements.txt . RUN pip install --no-cache-dir -r requirements.txt COPY app.py . EXPOSE 5000 CMD
["python", "app.py"]
```

Analyse Détaillée des Instructions Primitives

- **FROM :** Définit l'image de base (image parent). C'est obligatoirement la première instruction non-commentaire.
- **WORKDIR :** Fixe le répertoire de travail courant à l'intérieur du conteneur pour toutes les instructions suivantes (`COPY`, `RUN`, `CMD`). Il crée le dossier s'il n'existe pas.
- **COPY :** Transfère des fichiers ou dossiers depuis la machine hôte (contexte de build) vers le système de fichiers de l'image.
- **RUN :** Exécute une commande système pendant la phase de construction pour installer des paquets ou préparer l'environnement. Chaque `RUN` crée une nouvelle couche.
- **EXPOSE :** Documentation indicative informant Docker et les administrateurs du port sur lequel l'application écoute. *Note : n'ouvre pas le port sur l'hôte !*
- **CMD :** Définit la commande par défaut exécutée lors du démarrage du conteneur. Elle peut être surchargée lors du `docker run`.

Distinction Essentielle : CMD vs ENTRYPOINT

L'instruction `CMD` fournit des arguments par défaut aisément remplaçables lors de l'exécution de la CLI. À l'inverse, `ENTRYPOINT` transforme le conteneur en un exécutable strict, en combinant ses paramètres fixes avec les arguments passés au runtime.

5. COMMENT CONSTRUIRE ET EXÉCUTER UNE IMAGE ?

Une fois le Dockerfile rédigé à la racine de votre projet, la construction s'effectue via la commande `docker build`.

La Commande de Construction (docker build)

```
$ docker build -t my-api:1.0 .
```

Décomposition des paramètres :

- `docker build` : Appelle le moteur de construction Docker (BuildKit).
- `-t my-api:1.0` : Tague l'image avec un nom (`my-api`) et une version (`1.0`).
- `.` : Spécifie le **contexte de build** (ici le répertoire courant). Tout le contenu de ce dossier est envoyé au démon Docker.

Schéma du Cycle de Vie Complet : Du Code au Conteneur

```
Code Source + Dockerfile
        |
        ▼  (docker build -t my-api:1.0 .)
Image Docker Personnalisée (Stockée dans le registre local / Docker Images)
        |
        ▼  (docker run -p 5000:5000 my-api:1.0)
Conteneur en Cours d'Exécution (Processus isolé écoutant sur le port 5000)
```

Commandes Utiles de Vérification et Test

```
# Lister les images locales disponibles sur la machine $ docker images # Analyser la structure détaillée et les métadonnées d'une image $ docker image inspect my-api:1.0 # Lancer le conteneur en arrière-plan avec nettoyage automatique à l'arrêt $ docker run -d --rm --name api-container -p 5000:5000 my-api:1.0
```

Gestion du Naming et Versionnage des Tags

Un tag valide suit la convention `nom-de-registre/projet/image:version` (ex: `docker.io/karwisoft/api:1.2.0`). L'utilisation du tag par défaut `latest` est fortement déconseillée en production car elle masque la version réelle du binaire déployé.

6. IMAGES, COUCHES (LAYERS) ET CACHE

Chaque instruction dans un Dockerfile (`FROM`, `RUN`, `COPY`) crée une couche en lecture seule (read-only layer). L'image finale est un empilement ordonné de ces couches.

Invalidation du Cache : Comprendre la Règle d'Or

Lors d'un re-build, Docker réutilise le cache pour chaque étape tant que l'instruction et les fichiers copiés n'ont pas changé. Dès qu'une couche est modifiée, **le cache est invalidé pour cette couche ET toutes les suivantes !**

Comparaison d'Organisation de Dockerfile

❌ **Organisation inefficace (Invalide le cache à chaque modif de code) :**

```
FROM python:3.12-slim WORKDIR /app COPY . . # <--- Si app.py change, cette couche change ! RUN pip install --no-cache-dir -r requirements.txt # <--- Re-télécharge TOUT !
```

✅ **Organisation optimisée (Réutilise le cache des dépendances) :**

```
FROM python:3.12-slim WORKDIR /app COPY requirements.txt . # <--- Ne change presque jamais RUN pip install --no-cache-dir -r requirements.txt # <--- CACHÉ ! (Gain immense) COPY . . # <--- Seul le code source mis à jour est copié
```

Analyse Détaillée de l'Empreinte Mémoire des Couches

Chaque couche générée calcule une somme de contrôle (hash SHA-256). Lorsque deux images partagent des couches initiales identiques, ces couches ne sont stockées qu'une seule fois sur le disque dur du serveur hôte, réduisant drastiquement l'espace requis.

> 💡 **Gain de Productivité**
>
> En séparant la copie des dépendances du code source, le temps de build passe de 3 minutes à moins de 2 secondes lors des modifications quotidiennes de code !

7. LE FICHIER .DOCKERIGNORE & SÉCURITÉ DU CONTEXTE

Lorsqu'on lance `docker build`, le client Docker envoie l'intégralité du répertoire courant (le contexte) au démon. Sans filtrage, des fichiers volumineux ou sensibles sont inutilement transmis et incorporés dans l'image.

Rôle du fichier .dockerignore

Similaire au fichier `.gitignore`, le fichier `.dockerignore` spécifie les dossiers et motifs de fichiers à exclure du contexte de build.

```
# Exemple de fichier .dockerignore complet .git .venv __pycache__ *.pyc .env .gitignore
README.md node_modules *.log dist/ coverage/
```

Pourquoi est-ce Critique pour la Sécurité et les Performances ?

- **Protection contre la fuite de secrets :** Empêche l'inclusion d'un fichier `.env` contenant des mots de passe ou des clés API dans l'image finale publique/privée.
- **Réduction du temps de transfert :** Évite de transférer des dossiers gigantesques comme `node_modules` ou `.venv` créés sur la machine hôte.
- **Prévention des conflits d'architecture :** Évite d'intégrer des binaires compilés pour l'architecture hôte (ex: macOS/Windows) dans le conteneur Linux.
- **Reproductibilité des builds :** Assure que la construction dans l'image repose uniquement sur les sources propres et non sur des artefacts de build locaux résiduels.

Erreurs Classiques en Entreprise

L'absence de `.dockerignore` conduit fréquemment à la publication involontaire de clés SSH privées ou de jetons d'accès d'intégration continue dans des registres d'images publics, constituant une faille de sécurité majeure.

8. TECHNIQUES DE RÉDUCTION DE LA TAILLE DES IMAGES

Des images volumineuses (ex: > 1 Go) ralentissent le stockage, augmentent le temps de déploiement sur Kubernetes/Cloud et étendent la surface d'attaque.

1. Choisir la Bonne Image de Base

Image de Base	Taille Moyenne	Usage Recommandé
`python:3.12` (Ubuntu/Debian full)	~1.01 GB	Développement complexe nécessitant de nombreux compilateurs.
`python:3.12-slim` (Debian minimal)	~150 MB	**Recommandé** Excellent compromis légèreté/compatibilité.
`python:3.12-alpine` (Musl/Busybox)	~50 MB	Hyper léger, mais attention aux incompatibilités de compilation (wheels).

2. Bonnes Pratiques de Nettoyage dans le Dockerfile

- Ne pas installer d'outils de debug inutile en production (`curl`, `vim`, `net-tools`).
- Utiliser le fan-out `&&` pour nettoyer les caches dans la **MÊME** instruction `RUN` :

```
RUN apt-get update && apt-get install -y --no-install-recommends curl && rm -rf /var/lib/apt/lists/*
```

Si le nettoyage de `/var/lib/apt/lists/*` était exécuté dans un second bloc `RUN`, les fichiers supprimés resteraient stockés de manière permanente dans la couche précédente de l'image !

3. Introduction au Multi-Stage Building

La technique du Multi-Stage Build permet de séparer l'environnement de compilation (contenant les compilateurs lourd) de l'environnement d'exécution final, produisant des images d'une légèreté imbattable.

9. SÉCURITÉ DES CONTENEURS : LES 4 PRINCIPES CLÉS

Principe 1 — Ne Jamais Exécuter en Tant que ROOT !

Par défaut, un conteneur s'exécute sous le compte `root` (UID 0). En cas d'évasion de conteneur (container escape), l'attaquant devient super-utilisateur de la machine hôte.

```
# Créer et basculer sur un utilisateur non-privilégié RUN useradd --create-home appuser USER appuser
```

Principe 2 — Proscrire le Stockage de Secrets dans l'Image

Ne JAMAIS utiliser la directive `ENV` pour coder en dur des identifiants :

```
❌ A PROSCRIRE : ENV DB_PASSWORD="SuperSecretPassword123"
```

Tout utilisateur ayant accès à l'image peut extraire ces variables via `docker history` ou `docker inspect`. Injecter les secrets au runtime via des variables d'environnement ou Docker Secrets.

Principe 3 & 4 — Mises à jour & Privilèges Réseaux

- **Images de confiance :** Utiliser des images officielles vérifiées (Docker Official Images) et scanner les vulnérabilités CVE avec `docker scout` ou `trivy`.
- **Pas de mode privilégié :** Éviter absolument le drapeau `--privileged` qui désactive l'isolation du noyau et accorde l'accès complet aux périphériques physiques de l'hôte.

Analyse des Scans de Vulnérabilités (DevSecOps)

L'intégration d'outils d'analyse statique dans la chaîne d'assemblage permet d'interdire automatiquement la mise en production de tout conteneur présentant des failles de sécurité de niveau "Critical" ou "High".

10. INTÉGRATION NATIVE AVEC DOCKER COMPOSE

Plutôt que d'exécuter manuellement `docker build` puis `docker run`, Docker Compose permet de déclarer la phase de construction directement dans le fichier `compose.yaml`.

Exemple de Fichier compose.yaml avec Directive Build

```
services: backend: build: context: ./backend dockerfile: Dockerfile image: my-company/backend-api:1.0 ports: - "5000:5000" environment: - FLASK_ENV=production restart: always database: image: postgres:16-alpine environment: POSTGRES_PASSWORD_FILE: /run/secrets/db_password
```

Commandes de Workflow Unifié

- `docker compose up --build -d` : Forcer la re-construction des images modifiées avant de démarrer l'ensemble des services en arrière-plan.
- `docker compose logs -f backend` : Visualiser le flux de logs en direct du service construit.
- `docker compose down --volumes` : Arrêter proprement les conteneurs et supprimer les réseaux temporaires.

Gestion des Environnements Multiples (Dev / Prod)

En couplant Docker Compose avec des fichiers de surcouche (ex: `compose.override.yaml`), l'équipe peut basculer d'un mode de développement avec rechargement à chaud (hot-reload via volumes) à un mode de production immuable sécurisé.

TP 6 — DÉPLOIEMENT ET SÉCURISATION D'UNE API (1/2)

11. Architecture du Projet à Réaliser

```
docker-advanced-lab/ ├── Dockerfile ├── .dockerignore ├── requirements.txt ├── app.py └── compose.yaml
```

Étape 1 & 2 — Création de l'Application & du Dockerfile

Dans `app.py` :

```
from flask import Flask app = Flask(__name__) @app.get("/") def home(): return {"message": "My Docker API is running"} @app.get("/health") def health(): return {"status": "UP"} if __name__ == "__main__": app.run(host="0.0.0.0", port=5000)
```

Dans `Dockerfile` :

```
FROM python:3.12-slim WORKDIR /app COPY requirements.txt . RUN pip install --no-cache-dir -r requirements.txt COPY app.py . EXPOSE 5000 CMD ["python", "app.py"]
```

Questions de Réflexion & Diagnostic

ol>

- Pourquoi utiliser l'instruction `WORKDIR /app` au lieu de exécuter des commandes dans le dossier racine de l'image ?
- Pourquoi est-il indispensable de copier `requirements.txt` avant de copier le reste du code source `app.py` ?

Instructions d'Exécution de la Première Phase

Lancez le build avec la commande `docker build -t docker-advanced-api:1.0 .` puis vérifiez la présence de l'image compilée dans le registre local via `docker images`.

TP 6 — DÉPLOIEMENT ET SÉCURISATION D'UNE API (2/2)

Étape 5 & 6 — Ajout du .dockerignore & Test du Cache

Créer le fichier `.dockerignore` :

```
.git .venv __pycache__ *.pyc .env
```

Exercice de vérification du cache : Modifier une ligne dans `app.py` et relancer `docker build -t docker-advanced-api:1.0 .`. Observer que l'étape `RUN pip install` utilise `CACHED`.

Étape 8 — Durcissement Sécurité (User Non-Root)

Mettre à jour le `Dockerfile` pour basculer sous un utilisateur non-privilégié :

```
FROM python:3.12-slim WORKDIR /app COPY requirements.txt . RUN pip install --no-cache-dir -r requirements.txt && useradd --create-home appuser COPY --chown=appuser:appuser app.py . USER appuser EXPOSE 5000 CMD ["python", "app.py"]
```

Validation Finale de la Sécurité du Conteneur

Exécuter la commande `docker exec -it advanced-api whoami` et vérifier que le système répond `appuser` et non `root`.

Test des Endpoints Réseau de l'API

Validez le bon fonctionnement de l'application sécurisée à l'aide des commandes cURL suivantes :

```
$ curl http://localhost:5000/ {"message": "My Docker API is running"} $ curl http://localhost:5000/health {"status": "UP"}
```

13. EXERCICE AUTONOME & 14. MINI-PROJET

13. Exercice Autonome — Application Node.js ou Go

Chaque étudiant doit créer de zéro une seconde image pour une petite application de son choix avec les contraintes suivantes :

- Fichier `Dockerfile` fonctionnel et nettoyé.
- Inclusion obligatoire d'un fichier `.dockerignore`.
- Utilisation d'un utilisateur non-root (`USER node` ou `USER appuser`).
- Vérification du fonctionnement des logs et de la santé de l'application.

14. Cahier des Charges du Mini-Projet "Production-Ready Container"

L'objectif est de livrer un dépôt Git complet prêt pour la production comprenant :

1. Une application Web fonctionnelle avec endpoint de Healthcheck (`/health`).
2. Un Dockerfile optimisé et documenté.
3. Une configuration Docker Compose (`compose.yaml`).
4. Un fichier `README.md` exhaustif expliquant le build, l'exécution, les choix de sécurité et le diagnostic.

Le projet doit pouvoir être exécuté sur n'importe quelle machine disposant de Docker en lançant simplement la commande `docker compose up --build`.

Format de Livraison du Mini-Projet

Les livrables doivent être soumis sous la forme d'un lien vers un dépôt Git public ou d'une archive compressée `.zip` structurée, respectant scrupuleusement la nomenclature officielle des fichiers demandés.

15. BARÈME D'ÉVALUATION & PERSPECTIVES

Grille d'Évaluation Officielle (/20)

Compétence Évaluée	Barème	Critères d'Excellence
Compréhension du Dockerfile	3 pts	Directives adéquates, syntaxe propre et lisible.
Construction & Identification d'image	3 pts	Tags explicites, build sans erreur.
Gestion du Cache & .dockerignore	3 pts	Séparation intelligente des couches, exclusions correctes.
Fonctionnement du conteneur	2 pts	Port exposé, logs accessibles, conteneur stable.
Optimisation de l'image	2 pts	Choix d'image slim/alpine, suppression des fichiers temporaires.
Bonnes pratiques de sécurité	3 pts	Utilisateur non-root, aucun secret en dur.
Intégration Docker Compose	2 pts	Fichier compose.yaml propre avec directive build.
Documentation README.md	2 pts	Explications claires, justification des arbitrages.

16. Prochaine Étape : Module 7 — Pipelines CI/CD

La maîtrise du build d'images Docker est le prérequis fondamental pour aborder l'automatisation dans le module suivant :

```
Module 5: Compose ⟶ Module 6: Docker Avancé ⟶ Module 7: CI/CD (GitHub Actions)
```

Au Module 7, vous automatiserez le lancement de vos tests, la construction de vos images Docker et leur push automatique sur Docker Hub au sein d'un pipeline GitHub Actions !

Synthèse Globale du Cursus de Formation

En validant ce sixième module, l'apprenant franchit une étape décisive : il maîtrise désormais la création d'artefacts logiciels immuables, légers et hautement sécurisés, prêts à alimenter les chaînes de déploiement continu les plus exigeantes.